CS 312 — Assignment 3: Optimization 2

Sandra Yang (`aleyang`)

Setup. Code: `experiments/a3_optimization/` on branch `a3-experiments` of the fork `birdhumming/assignments`. Runs: W&B project `assignments`, entity `aleyang-stanford-university`, tags `a3-*`. All training ran on Modal H100s, at most two at a time. The Problem 4 checkpoints are the course-supplied AdamW and SGD runs; we measured them and never retrained them.

How to read the numbers. "Loss" is validation loss in nats of the course d8 model (8 layers, width 512, batch 64, AdamW with $\beta = (0.9, 0.95)$, weight decay 0.1) unless a problem says otherwise. From Assignment 1, the seed to seed standard deviation of this model's final loss is about 0.002. A gap below 0.005 is noise. A gap of 0.01 is small but real. A gap of 0.1 separates two regimes.

1 Mode connectivity

(a) Setup and what we measured

Train the d8 model (AdamW, peak learning rate 3×10^{-3}, constant after a 3% warmup so the two branches see the same learning rate) for a shared prefix of x steps, save the model and optimizer state, then fork two branches that differ only in the order of the training data (data seeds 1 and 2) and train each for y more steps. For every (x, y) cell we evaluate on the same 64 validation batches: both endpoints, the 0.5/0.5 weight average, the average of the two models' predicted probabilities (the ensemble), and the full interpolation curve at $\alpha \in \{0, 0.25, 0.5, 0.75, 1\}$. The barrier is $B(x, y) = L(\text{average}) - \frac{1}{2}\big(L(\theta_1) + L(\theta_2)\big)$. The handout's rule is that averaging "works" when $B \leq 0.01$.

Prediction before running: a longer shared prefix should make averaging work, because the branches start in the same basin and a few hundred steps of different data order will not move them to another one. Branches from a random initialization ($x = 0$) should fail once they have trained long enough to pick different internal coordinates.

(b) Results

Table 1: Midpoint barrier $B(x, y)$ in nats. Rows: shared prefix x steps. Columns: branch length y steps. Bold cells fail the $B \leq 0.01$ criterion; the $(50, 250)$ cell passes by 0.003 and is marked with a dagger.

$x \backslash y$	25	50	100	250	500
0	−0.026	−0.034	−0.026	**+0.225**	**+0.590**
50	−0.106	−0.094	−0.081	+0.007†	**+0.141**
100	−0.100	−0.123	−0.104	−0.053	**+0.026**
250	−0.093	−0.109	−0.105	−0.085	−0.053
500	−0.076	−0.095	−0.093	−0.093	−0.078
1000	−0.060	−0.076	−0.084	−0.086	−0.086

Table 2: Mean endpoint loss (left), gain of the weight average over the mean endpoint (middle) and gain of the prediction ensemble over the mean endpoint (right). Positive gain means better than either branch on average.

	endpoint mean					average gain					ensemble gain				
$x \backslash y$	25	50	100	250	500	25	50	100	250	500	25	50	100	250	500
0	6.45	5.65	5.03	4.31	3.75	.03	.03	.03	−.23	−.59	.04	.07	.11	.11	.11
50	5.28	5.06	4.70	4.15	3.71	.11	.09	.08	−.01	−.14	.09	.10	.11	.11	.10
100	4.83	4.69	4.47	3.99	3.66	.10	.12	.10	.05	−.03	.09	.11	.11	.11	.10
250	4.24	4.15	3.98	3.75	3.58	.09	.11	.11	.09	.05	.08	.09	.10	.10	.09
500	3.72	3.71	3.66	3.58	3.49	.08	.10	.09	.09	.08	.07	.08	.08	.09	.08
1000	3.48	3.48	3.46	3.43	3.38	.06	.08	.08	.09	.09	.06	.07	.07	.08	.08

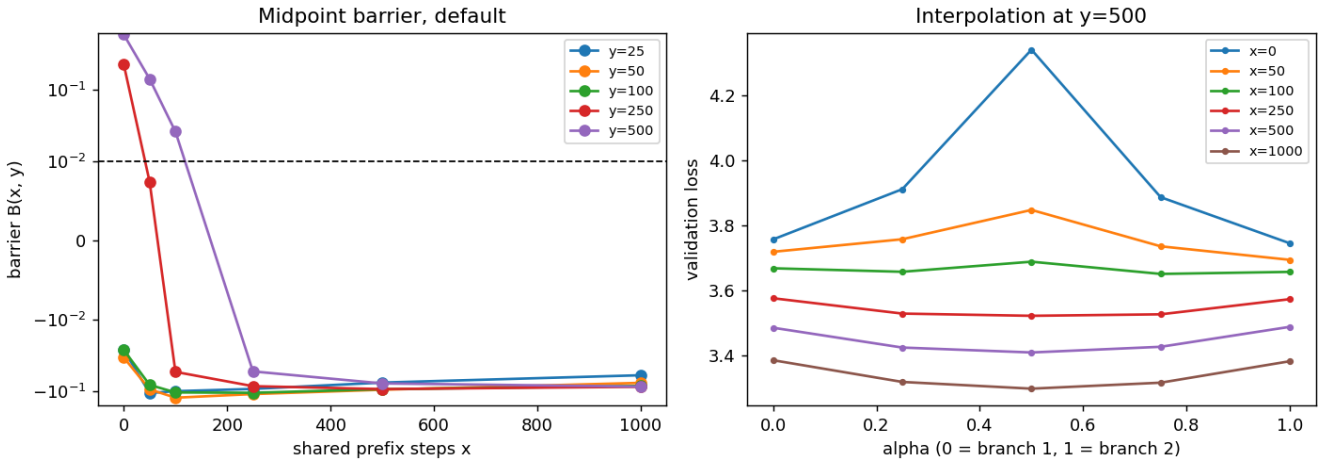


Figure 1: Left: barrier $B(x, y)$ against prefix length, one line per branch length, dashed line at the 0.01 criterion (symmetric log scale). Right: interpolation curves for the longest branches ($y = 500$).

Averaging works in 26 of 30 cells. It fails only when the prefix is short and the branches are long: $(0, 250)$, $(0, 500)$, $(50, 500)$ and $(100, 500)$. In the failing cells the midpoint is much worse than either endpoint, up to 0.59 nats at $(0, 500)$, and the interpolation curve has a clear hump (right panel, blue). Every other curve in that panel is flat or sags in the middle.

Two things stood out that the prediction did not cover.

First, the barrier is negative almost everywhere, and by a lot: around -0.1. The weight average is not just connected to the endpoints, it is better than both of them. The reason is the constant learning rate. Each branch is still bouncing across the river valley at the end of its y steps, and the two branches are at different points of their bounce because they saw different data. Their average sits lower in the valley than either one. This is exactly the parameter averaging effect from Problem 2(b), with two independent samples instead of a running EMA.

Second, when averaging works, it is about as good as ensembling. At $x \geq 250$ the average gain (0.05 to 0.11) matches or beats the ensemble gain (0.08 to 0.10) at one third of the inference cost. Ensembling never fails, because it only needs the two models to make different mistakes, not to live in the same basin; its gain is a steady 0.04 to 0.11 across the whole grid. Averaging needs the stronger condition and gives the larger reward when the condition holds.

The budget that buys connectivity is small. At $x = 250$ every branch length up to 500 passes, and the loss is still 3.5 to 4.2, far from converged. The first few hundred updates fix the coordinate system of the network: which neuron plays which role, which direction in the residual stream encodes what. Once that is fixed, data order changes the weights by amounts small enough that a straight line between the two branches stays in the valley. At $x = 0$ the two branches pick their coordinate systems independently and the straight line between them passes through a model that is neither.

(c) Ensemble versus average

The ensemble is better where averaging fails and no better where averaging works. In the failing $x = 0$ column the ensemble gains 0.11 while the average loses 0.23 to 0.59. At $x \geq 250$ both gain roughly 0.09. So the choice is: if you have a shared prefix, average the weights and pay for one model at inference; if you do not, ensemble.

(d) Does the training recipe change the basin?

Setup. The same 6×5 grid (prefix x, branch length y, two data orders) was rerun with one knob changed at a time: learning rate 1.5×10^{-3} or 6×10^{-3} instead of 3×10^{-3}, batch size 256 instead of 64, and linear decay instead of a constant learning rate. Results in Table 3 and Figure 2; the full grids are in `results/a3-mode-connectivity-*.json`.

Prediction, written down before the runs. Higher learning rate: more failing cells and bigger barriers, because the branches move further apart per step. Lower learning rate: fewer failures and a smaller averaging gain, since the branches bounce less. Batch 256: fewer failures, because the gradient noise that separates the two data orders is four times smaller. Linear decay: no change, because the schedule has decayed less than 16% of the way by step 1500 of 9375.

Table 3: Connectivity grid under recipe changes. "pass" counts cells with $B \leq 0.01$ out of 30; "first connected x" is the shortest prefix for which every branch length passes; gain is the median of (endpoint mean − average) over passing cells; $L(1000, 500)$ is the endpoint loss of the longest runs.

recipe	pass	first connected x	$B(0, 500)$	$B(50, 500)$	gain	$L(1000, 500)$
learning rate 1.5×10^{-3}	27	100	0.31	0.05	0.076	3.388
default (3×10^{-3}, batch 64, constant)	26	250	0.59	0.14	0.085	3.384
linear decay	26	250	0.61	0.13	0.080	3.369
learning rate 6×10^{-3}	24	250	1.20	0.32	0.091	3.448
batch 256	24	250	1.31	0.12	0.058	3.164

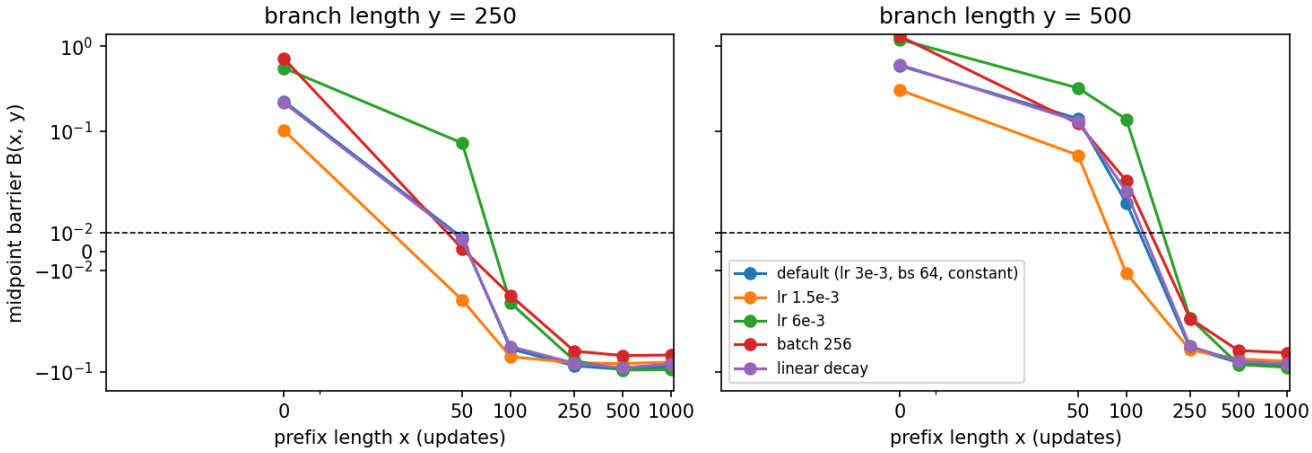


Figure 2: Midpoint barrier against prefix length for the two longest branch lengths, one line per recipe. Dashed line: the 0.01 criterion. Both axes are symmetric-log.

Three of the four predictions held. The learning rate sets the barrier almost proportionally: at $(0, 500)$ it is 0.31, 0.59 and 1.20 for 1.5, 3 and 6×10^{-3}, doubling each time the learning rate doubles. The low learning rate is the only recipe where a 100-step prefix is enough for every branch length, and its averaging gain is the smallest of the three (0.076 against 0.091), as expected from a smaller bounce. Linear decay reproduces the default grid cell for cell: the same four failures, barriers within 0.02.

The batch-size prediction was wrong, and in an informative way. Batch 256 has *more* failures than the default, not fewer, and the largest barrier in the whole experiment (1.31 at $(0, 500)$). Less gradient noise did not keep the branches together. What batch 256 did do is move each branch further per step: its endpoint loss after 1500 steps is 3.16 against 3.38, so a 500-step branch at batch 256 is a longer branch in units of progress. On that reading, $4\times$ the tokens per step behaved like $2\times$ the learning rate (1.31 against 1.20). The smaller averaging gain (0.058) is the one part of the batch-size prediction that held, and for the predicted reason: with less noise the two branches sit closer to the valley floor already, so their average has less to gain.

Pattern. The knob that controls whether two runs end in the same basin is how far the branches travel relative to how committed the prefix is, measured in optimization progress (learning rate times steps, or tokens), not how noisy the gradients are. Anything that moves the branches further (higher learning rate, bigger batch at fixed steps) needs a longer shared prefix; anything that moves them less (lower learning rate) needs a shorter one. The schedule is irrelevant at this horizon because it barely changes the step size. A prefix of 250 steps was enough in every recipe we tried, which says the coordinate-fixing phase is short and fairly robust.

Experiments that would test this. First, match branches by tokens or by loss instead of steps: run batch 256 with $y = 125$ and compare it to batch 64 with $y = 500$; if the barriers agree, progress is the right unit. Second, fix the branch recipe and only change the prefix recipe, to separate "how committed the prefix is" from "how far the branches go". Third, vary the prefix's data-order seed with the branch seeds fixed, to check that the prefix only needs to be long, not any particular sequence of data. Fourth, repeat the grid with momentum reset at the fork, since shared optimizer state is another way the two branches could be tied together.

2 River valley dynamics

(a) Schedules at equal tokens

Every run trains the d8 model on 614M tokens with 3% warmup. The schedules differ only in what happens after warmup. Linear decay, cosine decay and warmup-stable-decay with a 20% decay tail (wsd0.2) all go to zero at the end. The constant schedule stays at the peak.

Table 4: Final validation loss by schedule and peak learning rate. Best cell in bold.

Schedule	1.5×10^{-3}	3×10^{-3}	6×10^{-3}
Linear decay	2.9475	**2.9269**	2.9395
Cosine decay	2.9553	2.9339	2.9448
wsd0.2	2.9372	2.9362	2.9679
Constant	3.0609	3.0987	3.1839

Decaying the learning rate is worth 0.13 to 0.24 nats at the end, far more than any choice among the decay shapes (at most 0.02). The constant schedule gets worse as the learning rate rises, while the decayed schedules barely move. The constant run is not failing to learn. Its loss curve runs parallel to the others and just sits higher. That is the river valley picture: a large step size keeps the iterate bouncing across the valley walls, and the loss you measure includes the height of that bounce. Decay lets the iterate settle to the river bed.

The best reference for the rest of the problem is linear decay at 3×10^{-3}, loss 2.9269.

(b) Parameter averaging

We kept an exponential moving average (EMA) of the weights with decay 0.9, 0.99, 0.999 and 0.9999, updated every step and evaluated on the validation set alongside the raw weights. The EMA never fed back into training.

Table 5: Final validation loss of raw weights versus EMA weights, peak learning rate 3×10^{-3}.

Schedule	Raw	EMA 0.9	EMA 0.99	EMA 0.999	EMA 0.9999
Constant	3.0987	3.0609	2.9973	**2.9735**	4.43
Linear decay	**2.9269**	2.9269	2.9271	2.9309	4.39
wsd0.2	**2.9362**	2.9363	2.9364	2.9456	4.42

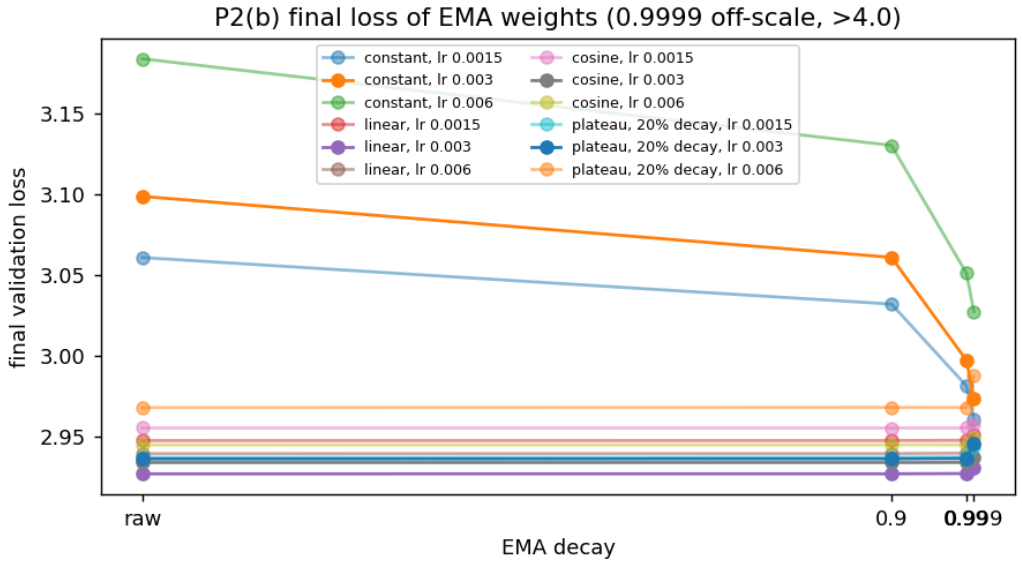


Figure 3: Final validation loss of the EMA weights as a function of the EMA decay, one line per run (raw weights at the left). The 0.9999 values are off scale above 4.

Averaging closes most of the gap but not all of it. EMA 0.999 on the constant run goes from 3.099 to 2.974, recovering 0.125 of the 0.172 gap to linear decay. The remaining 0.047 does not go away. On the decayed schedules the EMA gains nothing: EMA 0.99 is within 0.0002 of the raw weights and EMA 0.999 is 0.004 to 0.009 worse, because it still carries weights from before the decay finished. The decay already averaged out the bounce: a shrinking step size is itself a weighted average of recent iterates.

The decay constant has to match the time scale of the bounce. EMA 0.99 averages the last roughly 100 steps, which is too short to cancel oscillations that span several hundred steps, so it recovers 0.10 of the 0.17 gap where EMA 0.999 recovers 0.125. EMA 0.9999 averages the last 10,000 steps out of 9,375, so it is still dominated by early, bad weights and lands near 4.

Why does a gap remain? Averaging removes the bounce across the valley but does not move the iterate further along the river. A decayed learning rate does both: as the step shrinks, the iterate stops bouncing and also keeps descending along the valley floor with small, well aimed steps. The EMA can only report where the center of the bounce was.

(c) Learning rate dip

Prediction before running: the dip would lower the loss while the learning rate is small, and the gain would disappear once the learning rate returned to the plateau. The iterate drops to the river bed during the dip and gets kicked back up the valley walls afterward.

Setup: wsd0.2 at peak 3×10^{-3}. The dip run scales the learning rate by 0.1 between 40% and 50% of training, then rewarms to the plateau and follows the same decay tail as the control.

Table 6: Dip versus control at the key points. Validation loss, same tokens, same seed.

	Bottom of dip (step 4229)	Plateau after rewarm	Final
Control (wsd0.2)	3.189	reference	2.9362
Dip (0.1× over 40–50%)	**3.066**	+0.008 to +0.011	2.9394

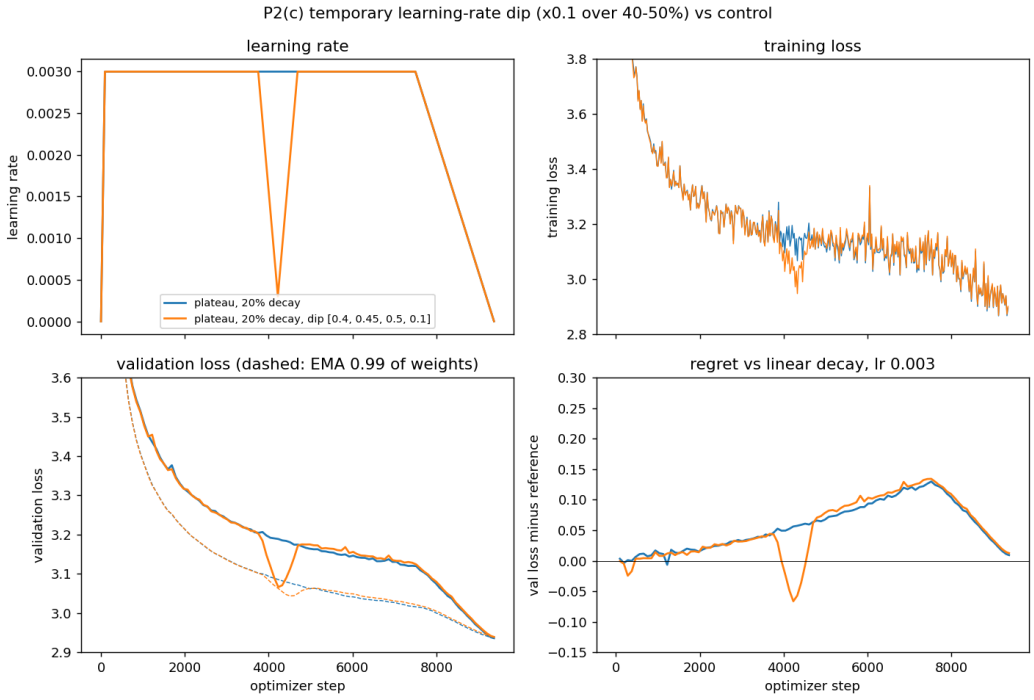


Figure 4: Learning rate and validation loss for the dip run and its control.

The prediction held. At the bottom of the dip the loss is 0.12 below the control, which is about the same size as the final gap between constant and decayed schedules in part (a). After rewarming, the dip run sits slightly above the

control for the rest of the plateau and finishes 0.003 worse. The two numbers are within noise of each other, so the honest statement is that the dip gained nothing by the end.

Intuition: the dip is a temporary decay. It shows where the river bed is but does not change where the iterate is along the river. When the learning rate returns to the peak, the iterate climbs the walls again and the ten percent of training spent at a small learning rate made less progress along the valley than the control did at full speed. The result is a slight deficit, not a gain. Lowering the learning rate helps only if you do not raise it again, which is why the decay tail belongs at the end.

(d) Horizon-free schedules

Two schedules that need no total budget: inverse square root decay after warmup, and cosine with warm restarts every 2000 steps (SGDR). Both at peak 3×10^{-3}, compared with the horizon aware linear decay from part (a).

Table 7: Horizon-free schedules against the budget aware baselines. Validation loss at 25%, 50% and 100% of 614M tokens, plus the EMA 0.999 value at the end.

Schedule	25%	50%	Final raw	Final EMA 0.999
Linear decay (knows the horizon)	3.2619	3.1110	**2.9269**	2.9309
Cosine decay (knows the horizon)	3.2693	3.1029	2.9339	2.9370
Inverse square root	**3.2538**	3.1270	3.0243	2.9900
Cosine warm restarts (2000 steps)	3.3913	3.2274	3.0394	2.9925

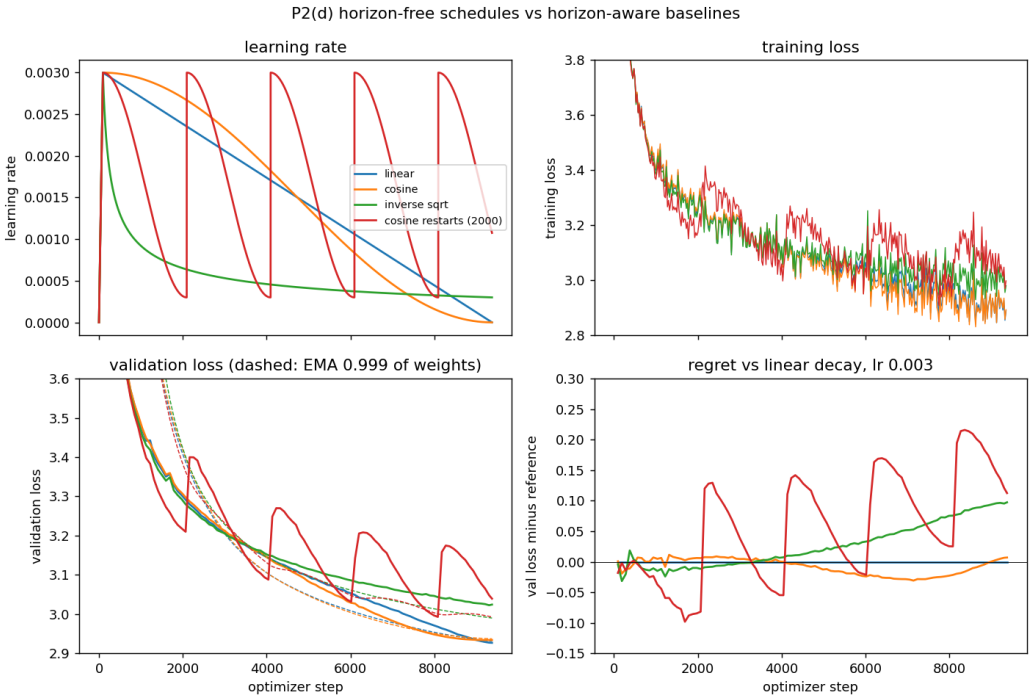


Figure 5: Learning rate and validation loss for the horizon-free schedules and the linear baseline.

Horizon-free schedules lose about 0.1 at the end (3.024 and 3.039 versus 2.927). Inverse square root is actually the best schedule at 25% of training (3.254 against 3.262 for linear and 3.269 for cosine), because it has already decayed a little while the others are still near their peak. By 50% it has fallen behind, and the gap grows through the decay tail. Warm restarts are worse at every checkpoint: each restart throws the iterate back up the valley walls, and the loss is measured right after a restart as often as right before one.

An EMA recovers most of the loss. EMA 0.999 brings inverse square root to 2.990 and warm restarts to 2.993, which leaves 0.06 on the table against linear decay. That is the same pattern as part (b): averaging takes care of the bounce, the missing decay tail takes care of the final descent, and only the schedule can do the latter.

The practical recipe when the horizon is unknown is inverse square root plus a weight EMA, and switch to a short linear tail the moment you learn when training will stop. The wsd0.2 result from part (a) says the tail only needs to be about 20% of training to get within 0.01 of the best schedule.

3 Edge of stability

(a) Full-batch gradient descent

Setup: 32 fixed sequences, plain gradient descent (no momentum, no weight decay, no gradient clipping), learning rate $\eta = 0.01$, 1,500 updates. Every 25 updates we measure $\lambda_{\max}$ of the Hessian on those 32 sequences and report $\eta\lambda_{\max}$. Gradient descent on a quadratic is stable only when $\eta\lambda_{\max} < 2$, so 2 is the dashed line.

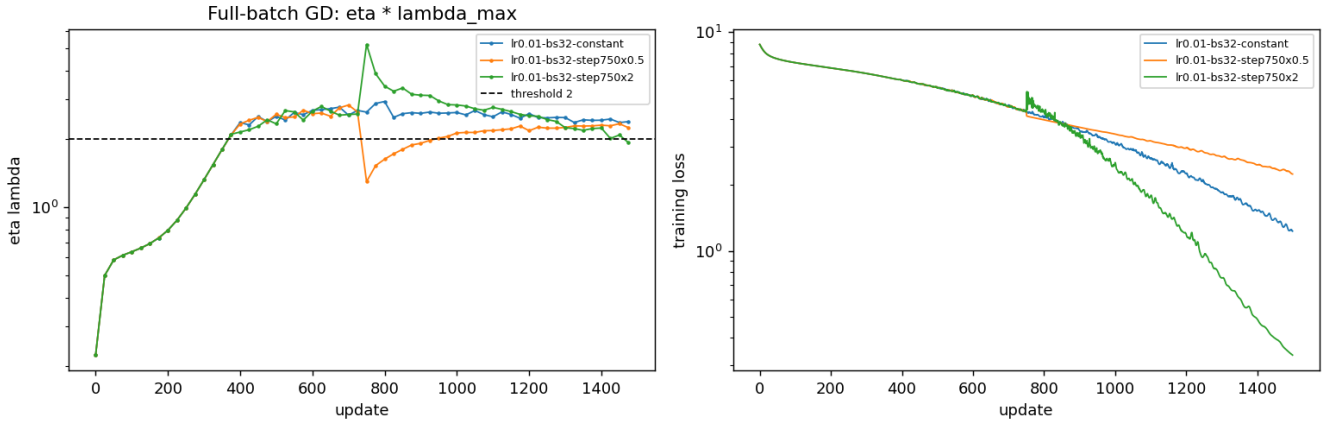


Figure 6: Left: $\eta\lambda_{\max}$ (log axis) for the three full-batch runs; dashed line at 2. Right: training loss. The learning rate is halved (orange) or doubled (green) at update 750.

Table 8: Full-batch runs. "Crossing" is the first measured update with $\eta\lambda_{\max} > 2$. "Plateau" is the median $\eta\lambda_{\max}$ from 100 updates after the crossing on. "Loss up" is the fraction of updates on which training loss rose, before and after the crossing.

Run	$\eta\lambda_{\max}$ at 0	Crossing	Plateau	Loss up before	Loss up after	Final loss
$\eta = 0.01$ constant	0.22	375	2.56	0.00	0.39	1.23
halve η at 750	0.22	375	2.24	0.00	0.24	2.25
double η at 750	0.22	375	2.70	0.00	0.41	0.33

Yes, sharpness goes to the edge and stays there. $\eta\lambda_{\max}$ starts at 0.22, rises for 375 updates (progressive sharpening), crosses 2, then hovers between 2.2 and 2.7 for the rest of training instead of continuing up. Before the crossing the loss falls on every single update. After it the loss rises on 24 to 41% of updates, the small oscillation that keeps $\lambda_{\max}$ pinned near $2/\eta$.

Halving η at 750 halves $\eta\lambda_{\max}$ to 1.3 at once, because $\lambda_{\max}$ does not change when η does. Then the curvature re-sharpens and is back at 2 by update 950. Doubling η sends $\eta\lambda_{\max}$ to about 6, which is unstable: the loss spikes, and the oscillation drives $\lambda_{\max}$ down over about 600 updates until $\eta\lambda_{\max}$ is back near 2.2. Both directions end at the same place. The sharpness tracks $2/\eta$, not the other way round.

The final losses (1.23, 2.25, 0.33) are training loss on 32 memorized sequences and say nothing about generalization. They do show that after the curvature adapts, the doubled learning rate trains much faster.

(b) Minibatch AdamW

Prediction: Adam's preconditioned sharpness $\eta\lambda_{\max}^{\text{pre}}$ should also rise to its stability threshold, which for $\beta_1 = 0.9$ is about 38, and hover there.

Setup: the d8 model with the usual AdamW recipe (batch 64, peak learning rate 3×10^{-3}, 3% warmup, constant afterwards) for 0.32 epochs of its 614M-token budget, which is 3,000 updates. Every 25 to 300 updates we estimated

$\lambda_{\max}$ of the preconditioned Hessian $P^{-1/2}HP^{-1/2}$, with P the current Adam second-moment estimate, on a fixed probe batch.

Table 9: Preconditioned sharpness $\eta\lambda_{\max}^{\text{pre}}$ of the minibatch AdamW run against the threshold 38.

update	16	32	48–208	500	1000	median 1526–2746	2700 (last)
$\eta\lambda_{\max}^{\text{pre}}$	1847	98	35–48	33.7	27.3	24.2	20.4
top eigvec. emb./MLP/attn.	1.0/0/0	.98/0/.02	.1/.5/.4	.04/.41/.53	.02/.40/.56		.04/.43/.48

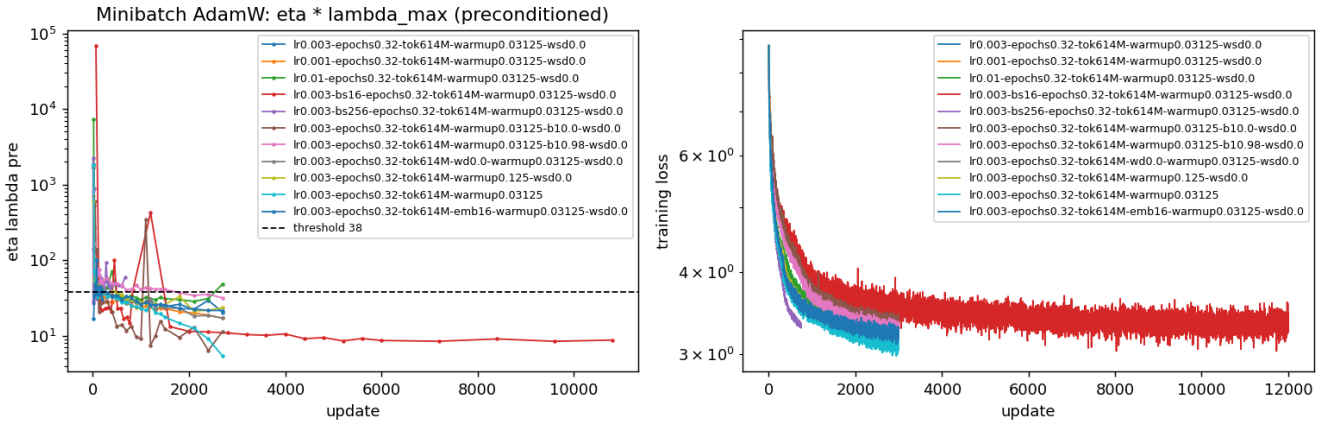


Figure 7: Left: $\eta\lambda_{\max}^{\text{pre}}$ on a log scale with the threshold at 38 (dashed). Right: training loss. The orange run is the 10^{-3} exploration run, still in progress at the time of writing.

The sharpness reaches the edge but does not stay there. During warmup (293 updates) $\eta\lambda_{\max}^{\text{pre}}$ is enormous (1847 at update 16, with the top eigenvector entirely in the embedding table, the same picture as the step-0 checkpoint in Problem 4) and then collapses onto the threshold: from update 48 to about 210 it oscillates between 35 and 48 around the line at 38, which is the Adam version of the edge of stability. At the same moment the top eigenvector moves out of the embedding and into the MLP and attention blocks, where it stays. After warmup ends and the learning rate is constant, it drifts down to 24 by the middle of the run and 20 at the end, well below the threshold, while the loss keeps falling smoothly.

Why minibatch Adam behaves differently from full-batch gradient descent. Full-batch gradient descent has nothing but the curvature to stop it, so it keeps sharpening until it hits the wall. Minibatch training adds gradient noise, and noise on a sharp direction is amplified by the curvature, so the optimizer is pushed off sharp regions before it reaches the deterministic threshold. The gap between 20 and 38 is how much room the noise claims. Adam adds a second effect: the preconditioner P tracks the gradient variance, which grows in exactly the directions that are sharp, so the preconditioned Hessian is flattened as training goes on. Both effects say that for a realistic language model run the stability threshold is a ceiling you touch during warmup, not a line you ride.

(c) Exploration

Setup. Ten minibatch AdamW runs that each change one thing from the 3(b) reference (614M tokens, constant learning rate after warmup, sharpness probed every 25 to 300 updates): peak learning rate 10^{-3} and 10^{-2}, batch size 16 and 256, $\beta_1 = 0$ and 0.98, no weight decay, 12.5% warmup instead of 3%, cosine decay instead of constant, and an embedding dimension cut to 16.

Prediction. If the threshold $38/\eta$ is what caps the preconditioned sharpness, the three learning rates should land on three different plateaus of $\lambda_{\max}^{\text{pre}}$ but the same plateau of $\eta\lambda_{\max}^{\text{pre}}$; $\beta_1 = 0$ lowers the threshold to about 2 (plain RMSProp) so that run should sit far lower in $\eta\lambda_{\max}^{\text{pre}}$; and larger batches reduce gradient noise and should push the run closer to the edge for longer. Cosine decay should let $\lambda_{\max}^{\text{pre}}$ grow as η falls (the quantity $\eta\lambda_{\max}^{\text{pre}}$ is what the edge constrains, not $\lambda_{\max}^{\text{pre}}$ alone).

Result. Only two knobs put the run on or over the edge: a large batch and a high learning rate. Everything else sits below it.

Table 10: Exploration runs. $\eta\lambda_{\max}^{\text{pre}}$ median over tokens 100M to 180M and at the last probe; threshold 38 for $\beta_1 = 0.9$, about 2 for $\beta_1 = 0$. Final training loss at update 3,000 (constant learning rate unless noted).

knob	value	median $\eta\lambda_{\max}^{\text{pre}}$	final $\eta\lambda_{\max}^{\text{pre}}$	final train loss
reference	lr 3×10^{-3}, batch 64	24.2	20.4	3.236
learning rate	10^{-3}	19.4	17.1	3.242
	10^{-2}	30.5	48.6	3.388
batch size	16	8.6	8.8	3.275
	256	47.7	59.9	3.335
β_1	0 (threshold ≈ 2)	10.5	11.5	3.369
	0.98	34.8	31.6	3.308
weight decay	0	18.4	17.1	3.202
warmup	12.5%	22.6	23.5	3.239
schedule	cosine decay	10.8	5.4	3.126
embedding dim	16	22.1	21.6	3.253

Learning rate behaves as predicted in direction but not in magnitude: tripling η from 10^{-3} to 3×10^{-3} raises $\eta\lambda_{\max}^{\text{pre}}$ only from 19 to 24, so $\lambda_{\max}^{\text{pre}}$ itself fell by about 2.4×. The optimizer does adapt curvature to the step size, just not all the way. At 10^{-2} the run ends above the threshold (48.6) and has the worst loss of the learning-rate sweep.

Batch size is the strongest knob, in the predicted direction. Batch 16 sits at 9, a quarter of the threshold; batch 256 sits at 48 to 60, above it for the whole run, with a worse loss than batch 64. Gradient noise is what keeps the small-batch run away from the edge: a noisy step already perturbs the iterate, so it never settles into the sharp directions long enough to be pushed out of them. With little noise the deterministic mechanism of Problem 3(a) takes over and the run rides the edge.

The $\beta_1 = 0$ run lands at 11, far below 38 but five times the RMSProp threshold of about 2, so the threshold for $\beta_1 = 0.9$ is not simply the one for $\beta_1 = 0$ scaled. Cosine decay refutes the prediction that $\lambda_{\max}^{\text{pre}}$ would grow as η shrinks: $\eta\lambda_{\max}^{\text{pre}}$ fell to 5 by the end, which is what the river-valley picture of Problem 2 wants (decay lets the iterate drop into the valley floor rather than re-sharpen). Removing weight decay gave the best constant-learning-rate loss and a lower sharpness, and the narrow embedding, the long warmup and $\beta_1 = 0.98$ changed little.

So the answer to "does sharpness approach and stabilize at the edge" for minibatch Adam is: it approaches during warmup, it stabilizes there only when the gradient is nearly deterministic (large batch), and otherwise it settles 20 to 75% below the threshold with the gap set mostly by batch size.

4 Inside the Hessian

(a) Curvature over training

We measured the Hessian of the course-supplied AdamW run at 13 checkpoints, from initialization to the end of its 614M-token schedule, on a fixed probe batch. Each measurement is a Lanczos run for the extreme eigenvalues, the top eigenvalue of each parameter family's diagonal block, the family composition of the top eigenvectors, and the Frobenius energy of every block pair.

Table 11: Extreme eigenvalues of the full Hessian over training. "Top family" is the parameter family that holds most of the top eigenvector's mass.

Step	Tokens	Probe loss	$\|g\|$	$\lambda_{\max}$	$\lambda_{\min}$	$\|\lambda_{\min}\|/\lambda_{\max}$	Top family
0	0	8.795	15.92	1431.5	−1410.1	0.985	embedding
3	0.2M	8.396	11.58	1571.7	−601.7	0.383	embedding
15	1.0M	6.922	0.86	53.4	−17.5	0.327	embedding
61	4.0M	5.401	0.62	18.1	−1.90	0.105	embedding
153	10M	4.607	0.27	6.50	−0.26	0.039	head
305	20M	4.109	0.19	4.33	−0.07	0.016	head
763	50M	3.574	0.12	3.02	−0.08	0.028	head
1526	100M	3.379	0.13	2.20	−0.06	0.029	head
3052	200M	3.220	0.11	3.16	−0.06	0.020	head
4578	300M	3.135	0.11	3.74	−0.05	0.013	head
6104	400M	3.063	0.12	4.16	−0.04	0.009	head
7630	500M	3.007	0.13	5.28	−0.05	0.009	head
9375	614M	2.947	0.13	5.97	−0.08	0.013	head

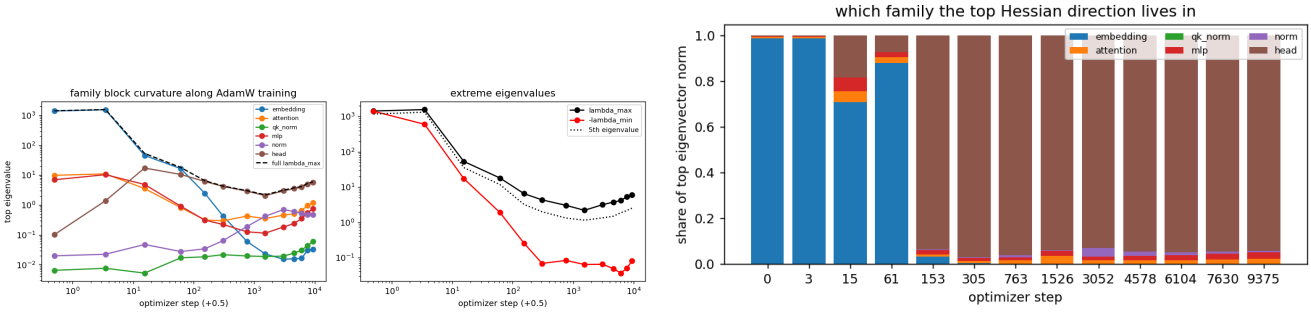


Figure 8: Left: $\lambda_{\max}$, $|\lambda_{\min}|$ and per-family block $\lambda_{\max}$ over training (log axes). Right: family composition of the top eigenvector.

Three phases. At initialization the Hessian is a saddle: $\lambda_{\min}$ is almost exactly $-\lambda_{\max}$ and both are about 1400, with 99% of the top eigenvector in the embedding table. Within 15 steps the embedding block's curvature collapses from 1414 to below 1, and by step 153 it is 0.02, a drop of five orders of magnitude. Negative curvature disappears at the same time: $|\lambda_{\min}|/\lambda_{\max}$ falls below 0.03 after step 153 and stays there.

From step 153 on, the top eigenvector lives in the output head (93 to 97% of its mass). $\lambda_{\max}$ keeps falling to a minimum of 2.2 at step 1526, then rises again to 6.0 by the end, a 2.7-fold increase during the linear-decay phase. That rise is the edge of stability seen from the other side: as the learning rate decays the optimizer can tolerate more curvature, and the curvature grows into the room it is given.

The gradient is nearly orthogonal to the top eigenvector after step 153 (overlap below 0.03). Progress happens in directions of low curvature; the sharp direction is where the iterate oscillates, not where it moves.

Trace and normalized sharpness. We also estimated $\mathrm{tr}(H)$ and $\mathrm{tr}(H^2)$ with 16 Hutchinson probes per checkpoint ($d = 34.6$M parameters).

Table 12: Hutchinson estimates. $\mathrm{tr}(H)/d$ is the mean eigenvalue (standard error in parentheses, from the 16 probes). Normalized sharpness is $\lambda_{\max}/(\mathrm{tr}(H)/d)$. The participation ratio $\mathrm{tr}(H)^2/(d\ \mathrm{tr}(H^2))$ is the fraction of directions that carry curvature; times d it is an effective number of curved directions.

Step	$\mathrm{tr}(H)/d$	$\mathrm{tr}(H)$	$\mathrm{tr}(H^2)/d$	$\lambda_{\max}$	$\lambda_{\max}/(\mathrm{tr}\,H/d)$	effective directions
0	-4.2×10^{-4} (1.4e-4)	−14,600	8.2	1431	—	0.8
3	3.8×10^{-3} (1.4e-4)	132,000	4.4	1572	4.1×10^{5}	115
15	6.2×10^{-4} (1.8e-6)	21,300	3.9×10^{-3}	53.4	8.7×10^{4}	3,400
61	1.0×10^{-4} (9.7e-7)	3,510	2.0×10^{-4}	18.1	1.8×10^{5}	1,800
153	3.0×10^{-5} (2.0e-7)	1,052	1.1×10^{-5}	6.50	2.1×10^{5}	2,900
763	1.5×10^{-5} (5.0e-8)	509	1.6×10^{-6}	3.02	2.1×10^{5}	4,700
1526	1.2×10^{-5} (5.2e-8)	422	1.0×10^{-6}	2.20	1.8×10^{5}	4,900
3052	1.3×10^{-5} (5.3e-8)	440	1.4×10^{-6}	3.16	2.5×10^{5}	4,000
6104	1.8×10^{-5} (6.6e-8)	632	2.8×10^{-6}	4.16	2.3×10^{5}	4,100
9375	2.7×10^{-5} (9.4e-8)	947	6.1×10^{-6}	5.97	2.2×10^{5}	4,300

At initialization the trace is negative: on average the loss curves down, not up, and the effective number of curved directions is below one, meaning a single embedding direction holds essentially all of $\mathrm{tr}(H^2)$. Three steps later the trace is +132,000, the largest value in the run, as the embedding block briefly becomes extremely sharp before AdamW flattens it.

After step 61 the trace follows the same U shape as $\lambda_{\max}$ (minimum 422 at step 1526, back to 947 at the end), and the ratio $\lambda_{\max}/(\mathrm{tr}\,H/d)$ stays between 1.8×10^5 and 2.5×10^5. The top eigenvalue is about 200,000 times the mean eigenvalue and that ratio hardly moves, so the spectrum shrinks and regrows as a whole rather than a few outliers growing on their own. The number of effective curved directions is about 4,000 to 5,000 out of 34.6 million for the whole second half of training. The Hessian is extremely low rank: the loss is curved in roughly one direction in ten thousand and flat everywhere else, which is why the projected updates of part (c) are an interesting experiment.

(b) Moving $\lambda_{\max}$ without moving the loss

Setup. Two ways to move $\lambda_{\max}$ at the final checkpoint. The first is a reparameterization that cannot change the loss: for a RMSNorm gain g feeding a linear layer W, the map $(g, W) \to (g/4,\ 4W)$ leaves every logit unchanged, because the normalized activation is multiplied by g and then by W. We verified on a small model in float64 that applying it to the attention input norms (feeding q, k, v), the MLP input norms (feeding gate and up), the final norm (feeding the head) or all three changes no logit (maximum $|\Delta\text{logit}| = 0.0$). The second way is to retrain the d8 model with a recipe change that ends in a flatter or sharper place: a constant learning rate instead of linear decay (wsd0.0), a 20% decay tail (wsd0.2), cosine decay, no weight decay, and a lower peak learning rate 1.5×10^{-3}.

Prediction before measuring. Scaling a weight by 4 divides its gradient by 4 and its Hessian block by 16; scaling the gain by 1/4 multiplies the gain's block by 16. The top eigenvector at the end is 94% in the output head, so the head rescaling should move $\lambda_{\max}$ by up to a factor of 16 in one direction or the other at exactly the same loss, and the attention and MLP rescalings should move it much less. For the retrained runs, the Problem 3 picture says $\lambda_{\max}$ settles near $38/\eta_{\text{final}}$ in preconditioned units, so the run that ends at the full learning rate (wsd0.0) should be the flattest and the run with the lowest peak learning rate the sharpest, while the final losses differ by the 0.01 to 0.17 seen in Problem 2.

Result: the reparameterization. The loss did not move at all and $\lambda_{\max}$ moved by at most 31%.

Table 13: Loss-invariant rescaling of the final AdamW checkpoint (gain ×1/4, linear ×4). Probe loss on a fixed 64-sequence batch; top eigenvector composition by family; $\lambda_{\max}$ of the diagonal block of the single tensor named.

rescaled pair	probe loss	$\|\nabla L\|$	$\lambda_{\max}$	top eigvec.	`lm_head` block	final-norm block
none (reference)	2.9474	0.130	5.97	94% head	5.64	0.30
attention norms → q, k, v	2.9474	0.120	5.88	96% head	5.64	0.30
MLP norms → gate, up	2.9474	0.123	6.46	75% head, 21% norm	5.64	0.30
final norm → head	2.9474	0.106	5.51	89% norm	0.35	4.79
all three	2.9474	0.084	7.82	97% norm	0.35	4.79
q projection alone	2.9474	0.128	5.97	94% head	5.64	0.30

The head prediction was half right. Multiplying `lm_head` by 4 did divide its block's curvature by 16 (5.64 to 0.35), exactly as the algebra says. But the gain it is paired with was divided by 4, so the final-norm block's curvature went up by 16 (0.30 to 4.79) and became the new top direction. The overall $\lambda_{\max}$ only fell from 5.97 to 5.51, and the top eigenvector flipped from 94% head to 89% final norm. Rescaling all three pairs stacked the same effect and made the model sharper (7.82). Scaling q alone did nothing because QK-norm removes the scale of q before it matters, so that direction is flat to begin with.

This is the honest answer to "can you move $\lambda_{\max}$ without moving the loss": yes, by any factor you like, because $\lambda_{\max}$ is a property of the coordinates and not of the function. Curvature is pushed from one side of a scale symmetry to the other. The gradient norm changes too (0.130 to 0.084), so neither $\lambda_{\max}$ nor $\|\nabla L\|$ is a coordinate-free diagnostic; only ratios that the optimizer actually sees, such as Adam's preconditioned sharpness, survive this.

Scale 4 overshoots. Balancing the two blocks needs $5.64/s^2 = 0.30\,s^2$, so $s \approx 2$ should give $\lambda_{\max} \approx 1.4$ at the same loss, a 4× drop. We ran that job after writing the prediction down. The two blocks landed where the algebra said (`lm_head` 1.41, final norm 1.20) but the full $\lambda_{\max}$ came out at 2.35, not 1.4, with the top eigenvector split 44% norm, 29% head, 19% MLP: once the two blocks are comparable, their off-diagonal coupling (the 28% cross-family energy from part (a)) carries the top direction. So the loss-invariant rescaling cut $\lambda_{\max}$ by 2.5× (5.97 to 2.35) at a probe loss of 2.9474 in both cases, and the block-diagonal estimate is a floor, not the answer.

Retrained variants. The second way to move $\lambda_{\max}$ is to train a different model. We measured the final checkpoint of five retrained d8 runs (the Problem 2 schedule runs plus a no-weight-decay run and a half-learning-rate run) with the same probe batches as the reference. Prediction, written before measuring: constant learning rate and no weight decay should both land at lower $\lambda_{\max}$, because the Problem 4(a) curve shows $\lambda_{\max}$ rising only during the decay phase, and weight decay is what keeps the norm gains and the head from shrinking their own curvature.

Table 14: Final-checkpoint Hessian of retrained d8 runs (614M tokens). Probe loss on the same batches as the rescaling table; reference is the course AdamW run (linear decay, lr 3×10^{-3}, wd 0.1).

run	probe loss	Δ loss	$\|\nabla L\|$	$\lambda_{\max}$	top eigvec.	head / norm / attn. block
reference (linear)	2.9474	–	0.130	5.97	94% head	5.64 / 0.30 / 0.5
cosine decay	2.9532	+0.006	0.118	5.62	95% head	5.34 / 0.52 / 1.12
plateau then decay (wsd0.2)	2.9592	+0.012	0.094	4.39	95% head	4.19 / 0.68 / 0.56
learning rate 1.5×10^{-3}	2.9674	+0.020	0.175	5.56	91% head	5.12 / – / 2.57
no weight decay	2.9751	+0.028	0.072	2.83	75% norm	1.71 / 2.33 / 0.50
constant learning rate	3.1231	+0.176	0.113	3.30	92% head	3.10 / 0.92 / 0.42

Both predictions held. Constant learning rate ends at $\lambda_{\max} = 3.3$, close to the value the reference had before its decay began (3.2 at step 3052), so the rise to 6 in the part (a) figure is the decay letting the iterate sink to the bottom of the valley, not training making the function sharper. But it costs 0.18 of loss, so it is not a way to "move $\lambda_{\max}$ without moving the loss". No weight decay halves $\lambda_{\max}$ (2.83) for +0.028 of loss, and the top direction moves from the head to the final RMSNorm gain, the same swap the rescaling produced: without decay the head weights grow, which is the $(g/s, sW)$ rescaling applied slowly by the optimizer. The plateau-then-decay schedule is the cheapest real change, −26% in $\lambda_{\max}$ for +0.012 of loss. Halving the learning rate barely moves $\lambda_{\max}$ at all.

Putting the two methods together: training knobs move $\lambda_{\max}$ by a factor of two at best and always move the loss along with it; a loss-invariant reparameterization moved it by 2.5× at exactly zero loss change. Any claim that compares $\lambda_{\max}$ across runs is therefore only meaningful if the runs share a parameterization and a weight-norm regime, and the fairer quantity to compare is the preconditioned sharpness of Problem 3, which divides out the scale that Adam divides out.

(c) Projected updates

Setup. Start from the course SGD-with-momentum checkpoint and continue training with three update rules: the full gradient step, the step projected onto the top k Hessian eigenvectors, and the step with that top subspace removed. The top eigenspace is recomputed every r steps with a Lanczos solver, and at every refresh we log the fraction of the gradient's squared norm that lies in the top subspace.

Prediction: the gradient fraction in the top subspace will be small (the Problem 4(a) overlap with the top eigenvector is below 0.03), so removing the top subspace should barely slow the loss, and keeping only the top subspace should

make almost no progress and may oscillate. This is the Gur-Ari "gradient descent happens in a tiny subspace" claim, tested directly.

Results. Starting point: the SGD-with-momentum checkpoint at step 1526 (validation loss 3.694, $\lambda_{\max} = 2.65$, $\eta = 0.85$, momentum 0.9, so $\eta\lambda_{\max} = 2.25$; the heavy-ball stability threshold is $2(1+\beta) = 3.8$). Each arm runs 300 updates with a fresh momentum buffer, $k = 10$ eigenvectors refreshed every 25 steps unless stated. All ten jobs are in Table 15; Figure 9 shows the three core arms and the $k = 50$ pair.

Table 15: Projected-update continuation of the SGD+momentum run. "top frac." is the fraction of the gradient's squared norm inside the current top-k subspace, averaged over the 300 steps / at the last step. $\lambda_{\max}$ at the first / last refresh.

update	start	k	refresh	lr scale	val. start → end	Δ val.	top frac. mean / last	$\lambda_{\max}$
full	1526	10	25	1	3.694 → 3.583	−0.112	0.093 / 0.100	2.65 → 2.96
top subspace only	1526	10	25	1	3.694 → 6.815	+3.121	0.174 / 0.137	2.65 → 16.8
top subspace removed	1526	10	25	1	3.694 → 3.582	−0.112	0.028 / 0.019	2.65 → 3.27
top only	1526	50	25	1	3.694 → 3.765	+0.070	0.151 / 0.115	2.65 → 2.86
removed	1526	50	25	1	3.694 → 3.579	−0.116	0.046 / 0.037	2.65 → 3.47
full	1526	10	25	0.3	3.694 → 3.579	−0.115	0.044 / 0.018	2.65 → 3.33
removed	1526	10	25	0.3	3.694 → 3.579	−0.116	0.016 / 0.013	2.65 → 3.41
removed	1526	10	5	1	3.694 → 3.584	−0.110	0.073 / 0.004	2.65 → 69.3
full	305	10	25	1	4.807 → 4.196	−0.611	0.178 / 0.085	3.35 → 3.14
removed	305	10	25	1	4.807 → 4.191	−0.615	0.049 / 0.017	3.35 → 3.96

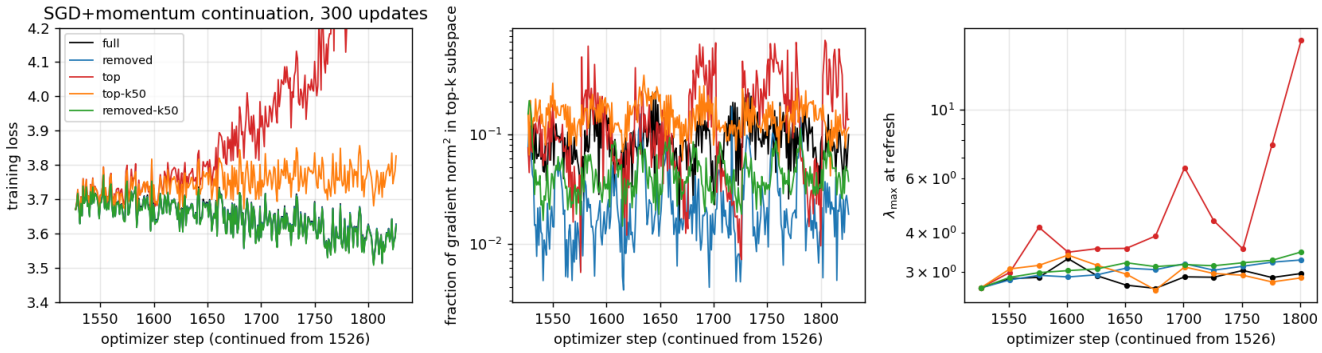


Figure 9: Left: training loss for the five arms started at step 1526 (the "full" curve is hidden under "removed"). Middle: fraction of the gradient in the current top-k subspace, log scale. Right: $\lambda_{\max}$ at each refresh.

The prediction held, and more strongly than expected. The top 10 of 34.6M directions hold about 9% of the gradient's energy under the full update (0.03 per direction when the overlap with the single top eigenvector in part (a) was below 0.03, so the next nine carry the rest). Removing them changes nothing: the loss after 300 steps is 3.582 against 3.583, and the two curves lie on top of each other in Figure 9. The same holds with $k = 50$, with a third of the learning rate, and from the step-305 checkpoint where the top fraction is twice as large (0.18 at the start): the "removed" arm is never worse than the full one, and is a hair better in four of five pairs. Progress lives in the bulk of low-curvature directions, as the trace in part (a) already suggested (4,000–5,000 effective curved directions, but gradient energy spread far wider than that).

Keeping only the top subspace does the opposite of training. With $k = 10$ the loss climbs from 3.69 to 6.8 and $\lambda_{\max}$ runs from 2.65 to 16.8, crossing the heavy-ball threshold ($\lambda_{\max} = 4.5$ at this η) around step 1700. The step projected onto the sharp directions is the oscillatory part of the update; without the drift along the valley floor that normally carries the iterate away from each bump, the oscillation feeds back on itself and sharpens the landscape until it diverges. With $k = 50$ the top subspace is large enough to contain a little useful signal (15% of the gradient), so it does not blow up, but it still drifts upward by 0.07.

The most useful side result is the refresh-every-5 run. Removing the top 10 directions every 5 steps instead of every 25 leaves the loss untouched (3.584) but lets $\lambda_{\max}$ grow steadily to 69, an $\eta\lambda_{\max}$ of 56 that would be instantly divergent for a full step. The curvature in the sharp directions is held in check only because the optimizer keeps stepping into them and bouncing: that is the edge-of-stability feedback loop of Problem 3 seen from the other side. Refresh every

25 steps leaves a few steps per cycle in which newly sharpened directions get a full update, which is enough to keep $\lambda_{\max}$ near 3; refresh every 5 steps removes that feedback and the sharpness runs away while the loss, which does not depend on those directions, is unaffected. Progress and stability are controlled by disjoint sets of directions.